Appraisal Management System — Phase 1 Implementation Plan

Project Name: Ardur Appraisal Management
Sprint Duration: 7 Days
Document Version: 1.0
Last Updated: December 10, 2025

Executive Summary

This document outlines the complete technical specification and implementation roadmap for building an internal web application designed specifically for mortgage underwriters. The system enables users to upload US residential appraisal documents (PDF format), automatically extract key data fields using OCR technology, perform quality control validations, and manage the review workflow with approval or revision actions.

The approach is intentionally pragmatic — we're building a functional MVP in one week, not a feature-complete enterprise solution. Every decision here favors getting something working and useful over perfection. Features that don't directly contribute to the core upload-OCR-review cycle are deliberately pushed to future phases.

Table of Contents

1. Project Vision and Goals
2. What We're Building (In-Scope)
3. What We're NOT Building (Out-of-Scope)
4. Technology Decisions and Rationale
5. System Architecture Overview
6. Security Model: Roles and Permissions
7. Data Model Specifications
8. API Contract Definitions
9. Day-by-Day Implementation Schedule
10. Definition of Done
11. Risk Assessment and Mitigation
12. Future Phase Considerations

Project Vision and Goals

The Problem We're Solving

Mortgage underwriters review dozens of appraisal documents daily. Today, this process involves manually opening each PDF, visually scanning for key fields (borrower name, property address, appraised value, etc.), cross-referencing values against loan documents, and flagging discrepancies. This is tedious, error-prone, and doesn't scale.

Our Solution

An internal web application that:

- Accepts appraisal PDF uploads from authenticated underwriters
- Automatically extracts key data fields using OCR
- Runs a rules engine to flag potential issues (missing values, validation failures, suspicious patterns)
- Presents everything in a clean interface where underwriters can review, comment, and approve or reject

Success Criteria for Phase 1

By the end of this sprint, we should have a working system where:

1. An underwriter can log in, upload an appraisal PDF
2. The system extracts basic fields (borrower name, property address, appraised value)
3. QC rules flag obvious issues (empty fields, invalid values)
4. The underwriter can view extracted data alongside the original PDF
5. The underwriter can approve the appraisal or mark it for revision, with comments
6. An admin can create new underwriter accounts

That's it. If we achieve this, Phase 1 is a success.

What We're Building (In-Scope)

Core Features — Must Have

Feature	Description	Priority
User Authentication	Email/password login with JWT tokens for API access. Session-based auth for web pages.	Critical
Role-Based Access Control	Two roles: Admin and Underwriter. Permissions enforced at API and UI levels.	Critical
PDF Upload & Storage	Accept multipart file uploads, validate PDF format, store files on local disk with structured naming.	Critical
OCR Integration	Call a Python-based OCR microservice that processes PDFs and returns structured JSON with extracted fields.	Critical
Quality Control Engine	Java-based rules engine that evaluates extracted data against validation rules and generates issue reports.	Critical
Appraisal List View	Dashboard showing all appraisals with filtering by status (Pending, Approved, Needs Revision).	Critical
Appraisal Detail View	Single page showing extracted data, QC issues, and the original PDF (embedded viewer or download link).	Critical
Review Workflow	Ability to change status from Pending to Approved or Needs Revision, with mandatory comment.	Critical
Review History	Track all status changes with timestamp, user, old status, new status, and comment.	Critical
User Management	Admin-only ability to list users and create new underwriter accounts.	High
Audit Trail	Basic created_at and updated_at timestamps on all records.	High

User Interface — Keeping It Simple

We're using **Spring Boot with Thymeleaf templates** — server-rendered HTML with Tailwind CSS for styling. This means:

- No separate frontend build process
- No REST API consumption from JavaScript (mostly)
- Instant compatibility with session-based auth
- Faster development velocity for a solo developer

If someone insists on React or Vue, they're welcome to migrate later. For a one-week sprint, server-rendered is the right call.

What We're NOT Building (Out-of-Scope)

Let's be crystal clear about boundaries. The following are explicitly deferred to future phases:

Feature	Why It's Out	Phase Target
Machine Learning Risk Scoring	Requires training data, model development, and significant infrastructure.	Phase 3+
NLP Analysis of Comments	Appraisal narratives require specialized NLP models.	Phase 3+
Computer Vision for Photo Analysis	Property photo analysis is a separate ML problem.	Phase 4+
Handwriting Recognition (ICR)	Handwritten annotations need specialized OCR.	Phase 3+
Multi-Tenant SaaS	Tenant isolation, billing, onboarding flows — significant complexity.	Phase 2+
LOS/AMS Integration	External system integrations require API contracts and vendor coordination.	Phase 2+
Complex Workflows	Escalation, multi-reviewer approval chains, SLAs, etc.	Phase 2+
Mobile App	Web-first. Mobile can come later.	Phase 4+
Offline Mode	Full connectivity assumed for Phase 1.	Phase 3+
Email Notifications	Nice-to-have, not essential for core workflow.	Phase 2
Reporting & Analytics	Business intelligence comes after we have data.	Phase 2

Technology Decisions and Rationale

Backend Stack

Component	Choice	Rationale
Language	Java 21	Modern LTS version with virtual threads, pattern matching, and record types. Team familiar with Java ecosystem.
Framework	Spring Boot 3.x	Industry standard for enterprise Java. Mature ecosystem, excellent documentation, built-in security.
Security	Spring Security + JWT	JWT for stateless API auth, session for web pages. Well-understood pattern.
Database	PostgreSQL 15+	JSONB support for OCR results, excellent performance, free and open source.
Build Tool	Maven	Simpler than Gradle for straightforward projects. Wide IDE support.
File Storage	Local Filesystem	For Phase 1, local storage is sufficient. Move to S3/GCS in Phase 2 if needed.

OCR Microservice

Component	Choice	Rationale
Language	Python 3.11+	Best ecosystem for OCR and document processing libraries.
Framework	FastAPI	Lightweight, fast, automatic OpenAPI docs, async support.
OCR Engine	Tesseract (via pytesseract)	Free, well-documented, good enough for typed text on forms.
Fallback	PaddleOCR	If Tesseract struggles with form layouts, PaddleOCR has better table detection.

Frontend Stack

Component	Choice	Rationale
Template Engine	Thymeleaf	Native Spring Boot integration, no build step, familiar HTML syntax.
CSS Framework	Tailwind CSS (CDN)	Rapid prototyping, utility-first, consistent design without writing custom CSS.
JavaScript	Minimal vanilla JS	Only for PDF viewer integration and form validations. No React/Vue/Angular.
PDF Viewer	PDF.js (Mozilla)	Free, powerful, embeddable. Show PDF alongside extracted data.

Why This Stack Works

1. **Unified codebase** — Backend and frontend in one Maven project. Deploy as single JAR.
2. **Minimal ops complexity** — Two services total: Java app + Python OCR. One database.
3. **Fast iteration** — Server-rendered pages mean change HTML → refresh browser → see result.
4. **Familiar patterns** — Standard Spring patterns, no surprising abstractions.
5. **Easy to hand off** — Any Java developer can understand this codebase.

System Architecture Overview

High-Level Component Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              BROWSER (User)                              │
│                                                                          │
│   ┌──────────────┐    ┌──────────────┐    ┌──────────────┐              │
│   │  Login Page  │    │  Dashboard   │    │ Appraisal Detail│           │
│   │              │    │ (List View)  │    │   (Review)   │              │
│   └──────┬───────┘    └──────┬───────┘    └──────┬───────┘              │
└──────────┼───────────────────┼───────────────────┼──────────────────────┘
           │                   │                   │
           ▼                   ▼                   ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                         SPRING BOOT APPLICATION                          │
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────┐       │
│   │                      Web Layer (Controllers)                 │       │
│   │   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐    │       │
│   │   │ Auth     │  │ Appraisal │  │ Admin    │  │ API      │    │       │
│   │   │ Controller│  │ Controller│  │ Controller│  │ (REST)   │    │       │
│   │   └────┬─────┘  └────┬─────┘  └────┬─────┘  └────┬─────┘    │       │
│   └────────┼─────────────┼─────────────┼─────────────┼──────────┘       │
│            │             │             │             │                  │
│   ┌────────▼─────────────▼─────────────▼─────────────▼──────────┐       │
│   │                       Service Layer                          │       │
│   │   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐    │       │
│   │   │ Auth     │  │ Appraisal│  │ OCR      │  │ QC       │    │       │
│   │   │ Service  │  │ Service  │  │ Client   │  │ Service  │    │       │
│   │   └────┬─────┘  └────┬─────┘  └────┬─────┘  └────┬─────┘    │       │
│   └────────┼─────────────┼─────────────┼─────────────┼──────────┘       │
│            │             │             │             │                  │
│   ┌────────▼─────────────▼─────────────┼─────────────▼──────────┐       │
│   │                  Repository Layer  │                        │       │
│   │   ┌──────────┐  ┌──────────┐       │  ┌──────────┐  ┌──────────┐    │       │
│   │   │ User     │  │ Appraisal│       │  │ QCIssue  │  │ Review   │    │       │
│   │   │ Repo     │  │ Repo     │       │  │ Repo     │  │ Action   │    │       │
│   │   └──────────┘  └──────────┘       │  └──────────┘  └──────────┘    │       │
│   └────────────────────────────────────┼────────────────────────────┘       │
│                                        │                                    │
└────────────────────────────────────────┼────────────────────────────────────┘
                                         │
                                         │ HTTP POST /ocr/appraisal
                                         ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                         PYTHON OCR MICROSERVICE                          │
│                                                                          │
│   ┌──────────────────────────────────────────────────────────────┐       │
│   │                        FastAPI Application                   │       │
│   │   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐      │       │
│   │   │ PDF Handler  │  │ Tesseract    │  │ Field        │      │       │
│   │   │ (Upload)     │  │ Engine       │  │ Extractor    │      │       │
│   │   └──────────────┘  └──────────────┘  └──────────────┘      │       │
│   └──────────────────────────────────────────────────────────────┘       │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘

                                         │
                                         ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                            POSTGRESQL DATABASE                           │
│                                                                          │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐               │
│   │  users   │  │ appraisals│  │ ocr_results│  │ qc_issues │               │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘               │
│   ┌──────────┐                                                           │
│   │review_   │                                                           │
│   │ actions  │                                                           │
```

```
│   └──────────┘                                                    │
│                                                                   │
└───────────────────────────────────────────────────────────────────┘
```

Package Structure (Java Backend)

```
com.ardur.appraisal
├── config/                 # Spring configuration classes
│   ├── SecurityConfig.java
│   ├── WebConfig.java
│   └── JwtConfig.java
├── auth/                   # Authentication module
│   ├── controller/
│   ├── service/
│   ├── dto/
│   └── jwt/
├── appraisal/              # Core appraisal module
│   ├── controller/
│   ├── service/
│   ├── repository/
│   ├── entity/
│   └── dto/
├── ocr/                    # OCR integration module
│   ├── client/
│   ├── dto/
│   └── exception/
├── qc/                     # Quality control module
│   ├── service/
│   ├── rules/
│   ├── entity/
│   └── repository/
├── admin/                  # Admin management module
│   ├── controller/
│   ├── service/
│   └── dto/
└── common/                 # Shared utilities
    ├── exception/
    ├── audit/
    └── util/
```

Request Flow — Upload and Review

Here's what happens when an underwriter uploads an appraisal and reviews it:

Upload Flow:

1. Underwriter clicks "Upload Appraisal" button on dashboard
2. Browser sends POST request with PDF file to `/api/v1/appraisals/upload`
3. Controller validates file type and size
4. AppraisalService saves PDF to `uploads/appraisals/{uuid}.pdf`
5. AppraisalService creates new Appraisal record (status = PENDING)
6. AppraisalService calls OcrClient to POST file to Python service
7. Python service extracts text, parses fields, returns JSON
8. AppraisalService saves AppraisalOCRResult with extracted JSON
9. QcService runs rules against extracted data
10. QcService saves QCIssue records for any failures
11. Response redirects to appraisal detail page

Review Flow:

1. Underwriter opens appraisal detail page
2. Page displays: extracted fields, QC issues, PDF viewer
3. Underwriter selects "Approve" or "Needs Revision"
4. Underwriter enters comment (required field)
5. Browser sends POST to `/api/v1/appraisals/{id}/status`
6. Service validates transition (PENDING → APPROVED or NEEDS_REVISION)

7. Service creates ReviewAction record
8. Service updates Appraisal status
9. Page refreshes showing new status

Security Model: Roles and Permissions

Role Definitions

ADMIN Role

- Full system access
- Can create, view, and delete user accounts
- Can view all appraisals regardless of assignment
- Can change any appraisal's status
- Can view audit logs and system metrics

UNDERWRITER Role

- Can upload new appraisals
- Can view appraisals (all for Phase 1, assigned-only in Phase 2)
- Can view OCR results and QC issues
- Can change status of appraisals they can view
- Can add review comments
- Cannot manage users or change roles

Permission Matrix

Action	Admin	Underwriter	Anonymous
View login page	✓	✓	✓
Authenticate	✓	✓	—
Upload appraisal	✓	✓	—
View appraisal list	✓	✓	—
View appraisal details	✓	✓	—
View OCR extracted data	✓	✓	—
View QC issues	✓	✓	—
Change appraisal status	✓	✓	—
Add review comments	✓	✓	—
View review history	✓	✓	—
List all users	✓	—	—
Create new user	✓	—	—
Delete user	✓	—	—
Change user role	✓	—	—
View audit logs	✓	—	—

Authentication Implementation

API Endpoints: JWT-based authentication

- Login returns access token (15 min expiry) and refresh token (7 day expiry)
- All API requests include `Authorization: Bearer <token>` header
- Token validated on each request, no server session required

Web Pages: Session-based authentication

- Spring Security form login
- Session stored server-side (in-memory for Phase 1, Redis in Phase 2)
- CSRF protection enabled
- Remember-me functionality via secure cookie

Password Requirements

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character

- BCrypt hashing with cost factor 12

Data Model Specifications

Entity Relationship Diagram

```
┌──────────────────────────────┐
│            users             │
├──────────────────────────────┤
│ id (PK, UUID)                │
│ email (UNIQUE, NOT NULL)     │
│ password_hash (NOT NULL)     │
│ role (ENUM: ADMIN, UNDERWRITER) │
│ first_name                   │
│ last_name                    │
│ is_active (DEFAULT true)     │
│ created_at                   │
│ updated_at                   │
└──────────────┬───────────────┘
               │
               │ 1:N (assigned_to)
               ▼
┌──────────────────────────────┐          ┌──────────────────────────────┐
│          appraisals          │          │      appraisal_ocr_results   │
├──────────────────────────────┤          ├──────────────────────────────┤
│ id (PK, UUID)                │ 1:1      │ id (PK, UUID)                │
│ file_path (NOT NULL)         │◄────────►│ appraisal_id (FK, UNIQUE)    │
│ original_filename            │          │ raw_text (TEXT)              │
│ borrower_name                │          │ extracted_json (JSONB)       │
│ property_address             │          │ confidence_score (FLOAT)     │
│ appraised_value (DECIMAL)    │          │ processing_status (ENUM)     │
│ loan_number                  │          │ error_message                │
│ form_type (e.g., 1004)       │          │ created_at                   │
│ status (ENUM)                │          └──────────────────────────────┘
│ assigned_to_user_id (FK)     │
│ created_at                   │          ┌──────────────────────────────┐
│ updated_at                   │          │          qc_issues           │
└──────────────┬───────────────┘          ├──────────────────────────────┤
               │                          │ id (PK, UUID)                │
               │ 1:N                      │ appraisal_id (FK)            │
               ▼                          │ rule_code (e.g., RULE-001)   │
┌──────────────────────────────┐          │ severity (ENUM: LOW/MED/HIGH) │
│        review_actions        │          │ field_name                   │
├──────────────────────────────┤          │ message (TEXT)               │
│ id (PK, UUID)                │          │ is_resolved (BOOLEAN)        │
│ appraisal_id (FK)            │          │ created_at                   │
│ user_id (FK)                 │          └──────────────────────────────┘
│ old_status (ENUM)            │                      ▲
│ new_status (ENUM)            │                      │ 1:N
│ comment (TEXT, NOT NULL)     │                      │
│ created_at                   │          ┌──────────┴───────────────────┐
└──────────────────────────────┘          │        (appraisals)          │
                                          └──────────────────────────────┘
```

Table Definitions

users

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique identifier
email	VARCHAR(255)	NOT NULL, UNIQUE	Login credential
password_hash	VARCHAR(255)	NOT NULL	BCrypt hash
role	VARCHAR(20)	NOT NULL, CHECK (role IN ('ADMIN', 'UNDERWRITER'))	Authorization level

Column	Type	Constraints	Description
first_name	VARCHAR(100)		Display name
last_name	VARCHAR(100)		Display name
is_active	BOOLEAN	NOT NULL, DEFAULT true	Soft delete flag
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Audit
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Audit

appraisals

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique identifier
file_path	VARCHAR(500)	NOT NULL	Path to uploaded PDF
original_filename	VARCHAR(255)		User's original filename
borrower_name	VARCHAR(255)		Extracted from OCR
property_address	TEXT		Extracted from OCR
appraised_value	DECIMAL(15,2)		Extracted value
loan_number	VARCHAR(50)		Reference number
form_type	VARCHAR(20)		Form identifier (1004, 1025, etc.)
status	VARCHAR(20)	NOT NULL, DEFAULT 'PENDING'	Workflow status
assigned_to_user_id	UUID	REFERENCES users(id)	Current owner
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Upload time
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Last modification

Status Values: PENDING, APPROVED, NEEDS_REVISION

appraisal_ocr_results

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
appraisal_id	UUID	NOT NULL, REFERENCES appraisals(id), UNIQUE	Parent appraisal
raw_text	TEXT		Full extracted text
extracted_json	JSONB		Structured field data
confidence_score	FLOAT		OCR confidence (0.0-1.0)
processing_status	VARCHAR(20)		PENDING, COMPLETED, FAILED
error_message	TEXT		If processing failed
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Processing time

qc_issues

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
appraisal_id	UUID	NOT NULL, REFERENCES appraisals(id)	Parent appraisal
rule_code	VARCHAR(20)	NOT NULL	Rule identifier
severity	VARCHAR(10)	NOT NULL, CHECK (severity IN ('LOW', 'MEDIUM', 'HIGH'))	Issue priority
field_name	VARCHAR(100)		Affected field
message	TEXT	NOT NULL	Human-readable explanation
is_resolved	BOOLEAN	NOT NULL, DEFAULT false	Manual override flag
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Detection time

review_actions

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique identifier
appraisal_id	UUID	NOT NULL, REFERENCES appraisals(id)	Parent appraisal
user_id	UUID	NOT NULL, REFERENCES users(id)	Reviewer
old_status	VARCHAR(20)	NOT NULL	Status before change
new_status	VARCHAR(20)	NOT NULL	Status after change
comment	TEXT	NOT NULL	Required explanation
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Action time

Database Indexes

```
-- Performance indexes
CREATE INDEX idx_appraisals_status ON appraisals(status);
CREATE INDEX idx_appraisals_assigned_to ON appraisals(assigned_to_user_id);
CREATE INDEX idx_appraisals_created_at ON appraisals(created_at DESC);
CREATE INDEX idx_qc_issues_appraisal ON qc_issues(appraisal_id);
CREATE INDEX idx_review_actions_appraisal ON review_actions(appraisal_id);
CREATE INDEX idx_users_email ON users(email);

-- Partial index for active users only
CREATE INDEX idx_users_active_email ON users(email) WHERE is_active = true;
```

API Contract Definitions

Authentication APIs

POST /api/v1/auth/login

Authenticate user and obtain JWT tokens.

Request:

```
{
  "email": "underwriter@example.com",
  "password": "SecurePass123!"
}
```

Response (200 OK):

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "refreshToken": "eyJhbGciOiJIUzI1NiIs...",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "user": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "underwriter@example.com",
    "firstName": "Jane",
    "lastName": "Smith",
    "role": "UNDERWRITER"
  }
}
```

Response (401 Unauthorized):

```
{
  "error": "INVALID_CREDENTIALS",
  "message": "Email or password is incorrect",
  "timestamp": "2025-12-10T14:30:00Z"
}
```

GET /api/v1/auth/me

Get current authenticated user information.

Headers:

```
Authorization: Bearer <accessToken>
```

Response (200 OK):

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "email": "underwriter@example.com",
  "firstName": "Jane",
  "lastName": "Smith",
  "role": "UNDERWRITER",
  "isActive": true,
  "createdAt": "2025-11-15T10:00:00Z"
}
```

POST /api/v1/auth/refresh

Refresh access token using refresh token.

Request:

```
{
  "refreshToken": "eyJhbGciOiJIUzI1NiIs..."
}
```

Response (200 OK):

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "tokenType": "Bearer",
  "expiresIn": 900
}
```

Appraisal APIs

POST /api/v1/appraisals/upload

Upload a new appraisal PDF document.

Headers:

```
Authorization: Bearer <accessToken>
Content-Type: multipart/form-data
```

Request (multipart/form-data):

- `file` (required): PDF file, max 50MB
- `loanNumber` (optional): Loan reference number

Response (201 Created):

```
{
  "id": "660e8400-e29b-41d4-a716-446655440001",
  "status": "PENDING",
  "originalFilename": "AppraisalReport_1234.pdf",
  "assignedTo": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "underwriter@example.com"
  },
  "createdAt": "2025-12-10T14:35:00Z",
  "message": "Appraisal uploaded successfully. OCR processing initiated."
}
```

Response (400 Bad Request):

```
{
  "error": "INVALID_FILE_TYPE",
  "message": "Only PDF files are accepted",
  "timestamp": "2025-12-10T14:35:00Z"
}
```

GET /api/v1/appraisals

List all appraisals with optional filtering.

Headers:

```
Authorization: Bearer <accessToken>
```

Query Parameters:

- `status` (optional): Filter by status (PENDING, APPROVED, NEEDS_REVISION)
- `page` (optional, default 0): Page number
- `size` (optional, default 20): Items per page
- `sortBy` (optional, default createdAt): Sort field
- `sortDir` (optional, default desc): Sort direction

Response (200 OK):

```
{
  "content": [
    {
      "id": "660e8400-e29b-41d4-a716-446655440001",
      "borrowerName": "John Doe",
      "propertyAddress": "123 Main Street, Anytown, NY 12345",
      "appraisedValue": 450000.00,
      "status": "PENDING",
      "formType": "1004",
      "qcIssueCount": 2,
      "assignedTo": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "email": "underwriter@example.com"
      },
      "createdAt": "2025-12-10T14:35:00Z"
    }
  ],
  "page": 0,
  "size": 20,
  "totalElements": 45,
  "totalPages": 3
}
```

GET /api/v1/appraisals/

Get detailed information for a specific appraisal.

Headers:

```
Authorization: Bearer <accessToken>
```

Response (200 OK):

```
{
  "id": "660e8400-e29b-41d4-a716-446655440001",
  "originalFilename": "AppraisalReport_1234.pdf",
  "filePath": "/api/v1/appraisals/660e8400-e29b-41d4-a716-446655440001/file",
  "borrowerName": "John Doe",
  "propertyAddress": "123 Main Street, Anytown, NY 12345",
  "appraisedValue": 450000.00,
  "loanNumber": "LN-2025-12345",
  "formType": "1004",
  "status": "PENDING",
  "assignedTo": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "underwriter@example.com",
    "firstName": "Jane",
    "lastName": "Smith"
  },
  "ocrResult": {
    "extractedFields": {
      "borrowerName": "John Doe",
      "propertyAddress": "123 Main Street, Anytown, NY 12345",
      "appraisedValue": 450000,
      "effectiveDate": "2025-12-01",
      "lenderName": "ABC Mortgage Corp",
      "checkboxes": {
        "isInFloodZone": false,
        "isForSale": false,
        "hasPoolOrSpa": true
      }
    },
    "confidenceScore": 0.87,
    "processingStatus": "COMPLETED"
  },
  "qcIssues": [
    {
      "id": "770e8400-e29b-41d4-a716-446655440002",
      "ruleCode": "RULE-003",
      "severity": "MEDIUM",
      "fieldName": "propertyAddress",
      "message": "Property address does not include ZIP+4 code",
      "isResolved": false
    }
  ],
  "reviewHistory": [
    {
      "id": "880e8400-e29b-41d4-a716-446655440003",
      "user": {
        "email": "admin@example.com",
        "firstName": "Admin"
      },
      "oldStatus": null,
      "newStatus": "PENDING",
      "comment": "Initial upload",
      "createdAt": "2025-12-10T14:35:00Z"
    }
  ],
  "createdAt": "2025-12-10T14:35:00Z",
  "updatedAt": "2025-12-10T14:36:00Z"
}
```


POST /api/v1/appraisals/

Change the status of an appraisal with a comment.

Headers:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```

Request:

```
{
  "status": "APPROVED",
  "comment": "All fields verified. Value is consistent with comparable properties. QC issues reviewed and acceptable."
}
```

Response (200 OK):

```
{
  "id": "660e8400-e29b-41d4-a716-446655440001",
  "status": "APPROVED",
  "updatedAt": "2025-12-10T15:00:00Z",
  "reviewAction": {
    "id": "990e8400-e29b-41d4-a716-446655440004",
    "oldStatus": "PENDING",
    "newStatus": "APPROVED",
    "comment": "All fields verified...",
    "createdAt": "2025-12-10T15:00:00Z"
  }
}
```

Response (400 Bad Request - Invalid Transition):

```
{
  "error": "INVALID_STATUS_TRANSITION",
  "message": "Cannot transition from APPROVED to PENDING",
  "timestamp": "2025-12-10T15:00:00Z"
}
```

GET /api/v1/appraisals/

Download the original PDF file.

Headers:

```
Authorization: Bearer <accessToken>
```

Response: Binary PDF file with appropriate Content-Type header.

Admin APIs

GET /api/v1/admin/users

List all users (Admin only).

Headers:

```
Authorization: Bearer <accessToken>
```

Response (200 OK):

```
{
  "content": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "email": "admin@example.com",
      "firstName": "Admin",
      "lastName": "User",
      "role": "ADMIN",
      "isActive": true,
      "createdAt": "2025-11-01T10:00:00Z"
    },
    {
      "id": "550e8400-e29b-41d4-a716-446655440010",
      "email": "underwriter@example.com",
      "firstName": "Jane",
      "lastName": "Smith",
      "role": "UNDERWRITER",
      "isActive": true,
      "createdAt": "2025-11-15T10:00:00Z"
    }
  ],
  "totalElements": 2
}
```

POST /api/v1/admin/users

Create a new user (Admin only).

Headers:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```

Request:

```
{
  "email": "new.underwriter@example.com",
  "firstName": "New",
  "lastName": "User",
  "role": "UNDERWRITER",
  "temporaryPassword": "TempPass123!"
}
```

Response (201 Created):

```
{
  "id": "550e8400-e29b-41d4-a716-446655440020",
  "email": "new.underwriter@example.com",
  "firstName": "New",
  "lastName": "User",
  "role": "UNDERWRITER",
  "isActive": true,
  "createdAt": "2025-12-10T16:00:00Z",
  "message": "User created successfully. Temporary password set."
}
```

OCR Service API (Python Microservice)

POST /ocr/appraisal

Process an appraisal PDF and extract fields.

Request (multipart/form-data):

- `file`: PDF file to process

Response (200 OK):

```
{
  "success": true,
  "processingTimeMs": 4523,
  "confidenceScore": 0.87,
  "formType": "1004",
  "extractedFields": {
    "borrowerName": "John Doe",
    "coBorrowerName": null,
    "propertyAddress": "123 Main Street, Anytown, NY 12345",
    "city": "Anytown",
    "state": "NY",
    "zipCode": "12345",
    "appraisedValue": 450000,
    "effectiveDate": "2025-12-01",
    "salePrice": 440000,
    "lenderName": "ABC Mortgage Corp",
    "appraiserName": "Licensed Appraiser Inc.",
    "appraiserLicenseNumber": "NY-12345"
  },
  "checkboxes": {
    "isInFloodZone": false,
    "isForSale": false,
    "hasPoolOrSpa": true,
    "isCondoOrPUD": false,
    "isManufacturedHome": false
  },
  "warnings": [
    "Low confidence on 'appraiserLicenseNumber' field (0.65)"
  ]
}
```

Response (400 Bad Request):

```
{
  "success": false,
  "error": "UNSUPPORTED_FORMAT",
  "message": "Could not parse PDF. File may be encrypted or corrupted."
}
```

Day-by-Day Implementation Schedule

Day 1: Project Foundation and Authentication

Morning (4 hours)

Start with a clean Spring Boot 3 project setup. Initialize with Maven, add essential dependencies: Spring Web, Spring Security, Spring Data JPA, PostgreSQL driver, Lombok, validation, and JWT libraries. Configure PostgreSQL connection in `application.yml` with separate profiles for dev and test environments.

Create the User entity with JPA annotations. Include all the fields we specified: UUID as primary key (let the database generate it), email with unique constraint, password hash, role enum, names, active flag, and audit timestamps. Add a corresponding UserRepository interface extending JpaRepository with a custom method to find by email.

Afternoon (4 hours)

Set up Spring Security configuration. For Phase 1, we'll use a hybrid approach: session-based auth for Thymeleaf web pages and JWT for REST API endpoints. Create the SecurityConfig class with two security filter chains — one for `/api/**` paths using JWT, one for web paths using form login.

Implement the JWT infrastructure: a JwtService that generates tokens using a secret key from configuration, validates incoming tokens, and extracts claims. Create an AuthController with endpoints for login, token refresh, and current user info. Write the AuthService that authenticates credentials against the database.

Set up Thymeleaf with Tailwind CSS (via CDN for simplicity). Create a base layout template and a login page. Wire up Spring Security's form login to redirect to a dashboard after successful auth.

Seed the database with an admin user and an underwriter user for testing. Use a CommandLineRunner or Flyway migration.

Deliverables:

- Runnable Spring Boot application
- PostgreSQL connected and initialized
- User entity and repository
- JWT token generation and validation
- Login API endpoint working
- Web login page functional
- Two test users seeded (admin, underwriter)

Day 2: Appraisal Entity and File Upload

Morning (4 hours)

Create the remaining entities: Appraisal, AppraisalOCRResult, QCIssue, and ReviewAction. Define the relationships: Appraisal has one OCRResult (one-to-one), multiple QCIssues (one-to-many), and multiple ReviewActions (one-to-many). Appraisal also references User for assignment.

Set up database migrations using Flyway. Write the initial migration script that creates all tables with proper constraints, foreign keys, and indexes. Run the migration to verify everything creates correctly.

Afternoon (4 hours)

Implement file upload functionality. Create a FileStorageService that handles saving uploaded PDFs to a structured directory: `uploads/appraisals/{year}/{month}/{uuid}.pdf`. Validate file type (must be PDF), check file size limits (50MB max), and generate unique filenames.

Build the AppraisalController with an upload endpoint. Accept multipart file uploads, call FileStorageService to save the file, create an Appraisal record in the database with status PENDING, and assign it to the current user.

Create the dashboard Thymeleaf template. Show a header with logged-in user info and logout button. Display a button or form to upload new appraisals. For now, after upload, just redirect to a placeholder detail page.

Deliverables:

- All entities created with JPA mappings
- Flyway migrations for all tables
- File storage service working
- Upload API endpoint accepting PDFs
- Dashboard page with upload form
- Appraisal records saving to database

Day 3: OCR Integration

Morning (4 hours)

Build the Python OCR microservice. Create a new directory `ocr-service/` with a FastAPI application. Install Tesseract system dependency. Add a single endpoint: POST `/ocr/appraisal` that accepts a PDF file.

For Day 3, the OCR parsing can be partially mocked. Implement basic PDF text extraction using PyMuPDF or pdf2image + Tesseract. Parse the raw text with regular expressions to extract obvious fields like borrower name (look for "Borrower:" pattern), property address, and appraised value.

Return structured JSON with extracted fields, checkboxes (hardcoded for now), and a confidence score. Include error handling for corrupted or password-protected PDFs.

Afternoon (4 hours)

Create the Java OcrClient class in the backend. Use Spring's WebClient to call the Python service. Handle connection failures gracefully — if OCR service is down, mark the appraisal's processing status as PENDING and allow manual retry later.

Wire up the OCR call into the upload flow. After saving the PDF and creating the Appraisal record, immediately call the OCR service. Parse the response and save an AppraisalOCRResult record. Update the Appraisal's extracted fields (borrowerName, propertyAddress, etc.) from the OCR data.

Add a "Process OCR" button on the detail page for cases where automatic processing failed. This calls an endpoint that retries OCR for a specific appraisal.

Deliverables:

- Python OCR service running on port 5001
- Basic PDF text extraction working
- Java client calling Python service
- OCR results saving to database

- Appraisal fields populated from OCR
- Manual OCR retry available

Day 4: QC Rules Engine and Detail View

Morning (4 hours)

Build the QC rules engine. Create a QcService with a `runRules(Appraisal, AppraisalOCRResult)` method. Design rules as separate classes implementing a Rule interface — each rule inspects the data and returns zero or one QCIssue.

Implement the Phase 1 rules:

- **RULE-001**: Appraised value missing or zero → HIGH severity
- **RULE-002**: Borrower name empty → MEDIUM severity
- **RULE-003**: Property address incomplete (missing city/state/zip) → MEDIUM severity
- **RULE-004**: "For sale" checkbox true but sale price missing → LOW severity
- **RULE-005**: Confidence score below 0.70 → LOW severity (suggest manual review)

Run all rules, save QCIssue records for failures.

Afternoon (4 hours)

Build the appraisal detail page in Thymeleaf. Layout:

- Left column (60%): Extracted data in clean table format, QC issues list with severity badges
- Right column (40%): Embedded PDF viewer using PDF.js, or download link as fallback

Show QC issues prominently with color-coded severity (red for HIGH, yellow for MEDIUM, gray for LOW). Include the rule code and message for each issue.

Create a simple endpoint `GET /api/v1/appraisals/{id}` that returns the complete detail including OCR result and QC issues.

Deliverables:

- QC rules engine with 5 initial rules
- QCIssue records created during processing
- Detail page showing extracted fields
- Detail page showing QC issues
- PDF viewer or download link working

Day 5: Review Workflow and Status Management

Morning (4 hours)

Implement status transition logic. Create an AppraisalStatusService that handles status changes with validation:

- PENDING → APPROVED (valid)
- PENDING → NEEDS_REVISION (valid)
- APPROVED → PENDING (invalid — no going back)
- NEEDS_REVISION → PENDING (invalid)

Add the status change endpoint: `POST /api/v1/appraisals/{id}/status`. Require a comment (not empty). Create a ReviewAction record capturing who changed what, when, and why.

Ensure only authenticated users can change status. For Phase 1, both Admin and Underwriter can change any appraisal.

Afternoon (4 hours)

Update the detail page with review controls:

- Two buttons: "Approve" and "Needs Revision"
- Text area for comment (required)
- Disable buttons if status is already final

Add a "Review History" section at the bottom of the detail page. Show all status changes in chronological order with user name, timestamp, and comment.

Build the appraisal list page. Show a table with columns: ID, Borrower, Address, Value, Status, QC Issues (count), Created Date. Add tabs or dropdown to filter by status: All, Pending, Approved, Needs Revision. Make rows clickable to navigate to detail page.

Deliverables:

- Status change API with validation
- ReviewAction records created

- Approve/Reject buttons on detail page
- Comment field required for status change
- Review history displayed
- Appraisal list with filtering

Day 6: Admin Features and Testing

Morning (4 hours)

Build admin user management:

- `GET /api/v1/admin/users` — list all users
- `POST /api/v1/admin/users` — create new user with temp password
- `PATCH /api/v1/admin/users/{id}/role` — change role (for demo purposes)

Create admin page in Thymeleaf. Show user list in a table. Add a simple form to create new underwriter. Only show this page to ADMIN role users.

Restrict these endpoints to ADMIN role using Spring Security's `@PreAuthorize("hasRole('ADMIN')")`.

Afternoon (4 hours)

Write automated tests. Focus on critical paths:

Unit Tests:

- QC rules produce correct issues for given inputs
- Status transition validation rejects invalid transitions
- JWT token generation and validation

Integration Tests (using `@SpringBootTest`):

- Login with valid credentials returns token
- Login with invalid credentials returns 401
- Upload requires authentication
- Upload saves file and creates record
- Status change creates ReviewAction
- Admin endpoints reject Underwriter role

Use test containers for PostgreSQL or H2 in-memory for speed.

Deliverables:

- Admin user list endpoint
- Admin create user endpoint
- Admin page in UI
- 10+ automated tests covering core flows
- Tests passing in CI

Day 7: Polish, Error Handling, and Documentation

Morning (4 hours)

Add proper error handling throughout:

- Global exception handler returning consistent JSON errors
- 400 for validation failures (with field-level messages)
- 401 for authentication failures
- 403 for authorization failures
- 404 for not found resources
- 500 with generic message (log details server-side)

Handle OCR service unavailability gracefully. If Python service is down, mark appraisal as OCR_PENDING and show appropriate message in UI.

Add logging: log every upload, OCR request, QC run, and status change. Use structured logging with request IDs for traceability.

Afternoon (4 hours)

Polish the UI:

- Consistent Tailwind styling throughout
- Status badges with colors (green/yellow/red)

- Loading indicators for async operations
- Success/error toast notifications
- Responsive layout (works on tablet/desktop)

Write documentation:

- README.md with project overview, setup instructions, and how to run
- ARCHITECTURE.md with system design and module descriptions
- API.md with endpoint documentation (can reference Swagger if you add it)
- DEPLOYMENT.md with deployment checklist

Add a "how to run locally" section that anyone can follow:

1. Install prerequisites (Java 21, PostgreSQL, Python 3.11)
2. Clone repo
3. Run database migrations
4. Start OCR service
5. Start Java application
6. Access at localhost:8080

Deliverables:

- Consistent error responses across all endpoints
- Logging for key operations
- Polished UI with proper styling
- Comprehensive documentation
- Local development setup instructions

Definition of Done

Phase 1 is complete when ALL of the following criteria are met:

Functional Requirements

- [ ] Admin can log in and access admin features
- [ ] Underwriter can log in and access appraisal features
- [ ] User can upload a PDF appraisal file
- [ ] System extracts text from PDF using OCR
- [ ] System parses extracted text into structured fields
- [ ] QC rules run automatically after OCR processing
- [ ] QC issues are displayed on the appraisal detail page
- [ ] User can view the original PDF alongside extracted data
- [ ] User can approve an appraisal with a comment
- [ ] User can mark an appraisal as needs revision with a comment
- [ ] Review history is visible on the detail page
- [ ] Admin can view list of all users
- [ ] Admin can create new underwriter accounts

Technical Requirements

- [ ] All API endpoints return proper HTTP status codes
- [ ] Authentication is required for all protected endpoints
- [ ] JWT tokens expire and can be refreshed
- [ ] Passwords are hashed with BCrypt
- [ ] File uploads are validated (type and size)
- [ ] Database schema is managed by migrations
- [ ] No hardcoded credentials in source code

Quality Requirements

- [ ] At least 10 automated tests covering critical paths
- [ ] All tests pass in clean environment
- [ ] No critical bugs in core workflow
- [ ] UI is usable on desktop browsers (Chrome, Firefox, Safari)
- [ ] Error messages are user-friendly

Documentation Requirements

- [ ] README with project overview
- [ ] Setup instructions for local development
- [ ] API documentation with example requests/responses
- [ ] Architecture diagram included

Risk Assessment and Mitigation

Technical Risks

Risk	Probability	Impact	Mitigation
OCR accuracy too low for production	Medium	High	Start with simple forms (1004); add confidence threshold warnings; plan for Phase 2 ML improvement
Python/Java integration issues	Low	Medium	Test integration early (Day 3); define clear API contract; add timeout handling
File storage runs out of space	Low	Medium	Monitor disk usage; plan S3 migration for Phase 2; implement cleanup for old files
JWT secret exposed	Low	Critical	Use environment variables; never commit secrets; rotate keys regularly

Schedule Risks

Risk	Probability	Impact	Mitigation
OCR field extraction takes longer	Medium	Medium	Accept partial extraction for Phase 1; hardcode form layout for 1004 only
UI polish takes too long	Medium	Low	Prioritize function over form; use Tailwind defaults; defer custom styling
Testing reveals major bugs	Medium	High	Start testing Day 5; fix critical bugs immediately; defer minor issues

Mitigation Strategies

1. **Daily Check-ins**: End each day with a brief status review. Adjust priorities if falling behind.
2. **Feature Cuts**: If behind on Day 5, cut admin user management (use SQL for user creation instead).
3. **Parallel Work**: OCR service can be developed independently while Java backend progresses.
4. **Incremental Value**: Each day's work should produce a demonstrable improvement, even if incomplete.

Future Phase Considerations

Phase 2 (Weeks 2-3)

- Email notifications for status changes
- Appraisal assignment workflow
- Enhanced QC rules (20+ rules covering more fields)
- Reporting dashboard (counts by status, turnaround times)
- S3 file storage migration
- Redis session storage

Phase 3 (Month 2)

- Multi-tenant support (separate customers)
- Advanced OCR (PaddleOCR for table extraction)
- ML-based risk scoring (value vs. comparables)
- Integration with external LOS systems
- Mobile-responsive UI improvements

Phase 4+ (Future)

- Computer vision for property photos
- NLP analysis of appraiser commentary
- Automated comparable property lookups
- Full audit trail with compliance reporting
- Custom rule builder for admin

Appendix A: Environment Setup

Prerequisites

- Java 21 (recommend using SDKMAN)
- PostgreSQL 15+
- Python 3.11+
- Node.js 18+ (for Tailwind build if needed)
- Docker (optional, for containerized services)

Quick Start Commands

```
# Clone the repository
git clone git@github.com:your-org/ardur-appraisal.git
cd ardur-appraisal

# Set up Python OCR service
cd ocr-service
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --port 5001 &

# Set up Java backend
cd ../backend
./mvnw spring-boot:run -Dspring.profiles.active=dev
```

Configuration Files to Create

application-dev.yml:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/appraisal_dev
    username: postgres
    password: local_dev_password
  jpa:
    hibernate:
      ddl-auto: validate
jwt:
  secret: ${JWT_SECRET:change-this-in-production}
  expiration: 900
  refresh-expiration: 604800
ocr:
  service-url: http://localhost:5001
file:
  upload-dir: ./uploads
```

Appendix B: QC Rules Reference

Code	Name	Severity	Condition	Message
RULE-001	Missing Value	HIGH	appraisedValue is null or 0	Appraised value is missing or invalid
RULE-002	Missing Borrower	MEDIUM	borrowerName is blank	Borrower name is not extracted
RULE-003	Missing Address	MEDIUM	propertyAddress incomplete	Property address is missing required components
RULE-004	Sale Price Mismatch	LOW	isForSale=true AND salePrice=null	Property marked for sale but no sale price found
RULE-005	Low Confidence	LOW	confidenceScore < 0.70	OCR confidence below threshold - manual review recommended

---

Document prepared for the Ardur Appraisal Management project. Contact the development team for questions or clarifications.

System Architecture

This document provides a detailed technical overview of the Ardur Appraisal Management System architecture, including component design, data flow, and integration patterns.

Table of Contents

1. Architecture Overview
2. Component Design
3. Data Flow Patterns
4. Security Architecture
5. Integration Patterns
6. Scalability Considerations

Architecture Overview

The system follows a **modular monolith** architecture for the Java backend with a separate Python microservice for OCR processing. This approach balances simplicity with clear separation of concerns.

High-Level View

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                                   CLIENTS                                    │
│  ┌──────────────────────┐  ┌──────────────────────┐                          │
│  │    Web Browser       │  │    API Consumer      │                          │
│  │  (Thymeleaf Pages)   │  │   (REST/JSON)        │                          │
│  └──────────────────────┘  └──────────────────────┘                          │
└──────────────────────────────────────────────────────────────────────────────┘
            │                          │
            │  HTTP(S)                 │  HTTP(S) + JWT
            ▼                          ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                     JAVA BACKEND (Spring Boot 3)                             │
│                                                                              │
│  ┌──────────────────────────────────────────────────────────────────────┐    │
│  │                      Presentation Layer                              │    │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐                │    │
│  │  │ Web          │  │ REST API     │  │ Error        │                │    │
│  │  │ Controllers  │  │ Controllers  │  │ Handlers     │                │    │
│  │  └──────────────┘  └──────────────┘  └──────────────┘                │    │
│  └──────────────────────────────────────────────────────────────────────┘    │
│                                  │                                           │
│  ┌──────────────────────────────────────────────────────────────────────┐    │
│  │                        Service Layer                                 │    │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐             │    │
│  │  │ Auth     │  │ Appraisal│  │ QC       │  │ Admin    │             │    │
│  │  │ Service  │  │ Service  │  │ Service  │  │ Service  │             │    │
│  │  └──────────┘  └──────────┘  └──────────┘  └──────────┘             │    │
│  └──────────────────────────────────────────────────────────────────────┘    │
│                                  │                                           │
│  ┌──────────────────────────────────────────────────────────────────────┐    │
│  │                     Infrastructure Layer                             │    │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐             │    │
│  │  │ JPA      │  │ File     │  │ OCR      │  │ JWT      │             │    │
│  │  │ Repos    │  │ Storage  │  │ Client   │  │ Provider │             │    │
│  │  └──────────┘  └──────────┘  └──────────┘  └──────────┘             │    │
│  └──────────────────────────────────────────────────────────────────────┘    │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘
            │                                  │
            │ JDBC                             │ HTTP
            ▼                                  ▼
┌──────────────────────────┐    ┌──────────────────────────┐
│       PostgreSQL         │    │     Python OCR Service   │
│                          │    │        (FastAPI)         │
│  ┌────┐ ┌────┐ ┌────┐    │    │  ┌────────┐ ┌────────┐   │
│  │usr │ │app │ │qc  │    │    │  │PDF     │ │Tesseract│  │
│  │    │ │    │ │    │    │    │  │Parser  │ │Engine  │   │
│  └────┘ └────┘ └────┘    │    │  └────────┘ └────────┘   │
│                          │    │                          │
└──────────────────────────┘    └──────────────────────────┘
```

Design Principles

1. **Layered Architecture** — Clear separation between presentation, business logic, and data access
2. **Dependency Injection** — All components wired via Spring's IoC container
3. **Single Responsibility** — Each service handles one domain area
4. **Fail-Safe Defaults** — Graceful degradation when OCR service is unavailable
5. **Stateless APIs** — JWT-based authentication, no server-side session for API calls

---

Component Design

Authentication Module (`auth/`)

Handles user identity and access control.

Class	Responsibility
`AuthController`	Login, logout, token refresh endpoints
`AuthService`	Credential validation, JWT generation
`JwtService`	Token creation, parsing, validation
`UserDetailsServiceImpl`	Spring Security user loading
`JwtAuthenticationFilter`	Intercepts requests, validates tokens

Key Design Decisions:

- Hybrid auth: Sessions for web pages, JWT for API
- Access tokens expire in 15 minutes (security vs convenience trade-off)
- Refresh tokens last 7 days, can be revoked
- BCrypt with cost factor 12 for password hashing

Appraisal Module (`appraisal/`)

Core business logic for appraisal document processing.

Class	Responsibility
`AppraisalController`	CRUD endpoints, file upload
`AppraisalService`	Business logic orchestration
`FileStorageService`	Save/retrieve PDF files
`AppraisalRepository`	Database operations
`AppraisalMapper`	Entity ↔ DTO conversions

Upload Flow:

1. Validate file type (must be PDF)
2. Validate file size (≤ 50MB)
3. Generate unique filename: `{year}/{month}/{uuid}.pdf`
4. Save file to disk
5. Create `Appraisal` entity with PENDING status
6. Trigger OCR processing asynchronously (Phase 1: synchronous)

OCR Integration Module (`ocr/`)

Communicates with the Python OCR microservice.

Class	Responsibility
`OcrClient`	HTTP calls to Python service
`OcrResponseParser`	Map JSON response to domain objects
`OcrRetryService`	Handle failures, implement retry logic

Error Handling Strategy:

- If OCR service times out (> 30s): Mark as `OCR_PENDING`, allow manual retry
- If OCR service returns error: Log details, mark as `OCR_FAILED`
- If connection refused: Queue for background retry (Phase 1: manual retry only)

QC Rules Engine (`qc/`)

Validates extracted data against business rules.

Class	Responsibility
`QcService`	Orchestrates rule execution
`Rule` (interface)	Contract for individual rules
`MissingValueRule`	Check required fields present
`ValueRangeRule`	Validate numeric ranges
`ConsistencyRule`	Cross-field validation

Rule Execution Pattern:

```
public interface Rule {
    Optional<QcIssue> evaluate(Appraisal appraisal, OcrResult ocr);
}

// Each rule returns Optional.empty() if passed, or QcIssue if failed
```

Phase 1 Rules:

Rule	Check	Severity
RULE-001	Appraised value > 0	HIGH
RULE-002	Borrower name present	MEDIUM
RULE-003	Property address complete	MEDIUM
RULE-004	Sale price when "for sale"	LOW
RULE-005	OCR confidence ≥ 0.70	LOW

Admin Module (`admin/`)

User management for administrators.

Class	Responsibility
`AdminController`	User CRUD endpoints
`AdminService`	User creation, role changes
`UserRepository`	Database operations

Security:

- All endpoints require `ROLE_ADMIN`
- Cannot delete yourself
- Cannot demote the last admin

Data Flow Patterns

PDF Upload to Review

```
User                    Controller          Service             External
 |                         |                   |                   |
 |── POST /upload ────────►|                   |                   |
 |    (multipart)          |                   |                   |
 |                         |── validate() ────►|                   |
 |                         |                   |                   |
 |                         |                   |── saveFile() ────►| Disk
 |                         |                   |◄──────── path ────|
 |                         |                   |                   |
 |                         |                   |── create() ──────►| DB
 |                         |                   |◄────── entity ────|
 |                         |                   |                   |
 |                         |                   |─ POST /ocr ──────►| Python
 |                         |                   |◄─── JSON ─────────|
 |                         |                   |                   |
 |                         |                   |── runRules() ────►| (internal)
 |                         |                   |                   |
 |                         |                   |── saveAll() ─────►| DB
 |                         |◄── 201 Created ───|                   |
 |◄── Redirect to detail ──|                   |                   |
```

Status Change

```
User                    Controller          Service
 |                          |                  |
 |- POST /status ----------->|                  |
 |  { APPROVED, comment }   |                  |
 |                          |- changeStatus() ->|
 |                          |                  |--- validate ------->
 |                          |                  |    transition
 |                          |                  |
 |                          |                  |--- create -------->
 |                          |                  |    ReviewAction
 |                          |                  |
 |                          |                  |--- update -------->
 |                          |                  |    Appraisal
 |                          |<--- 200 OK -------|
 |<--- Updated entity -------|                  |
```

Security Architecture

Authentication Flow

```
                          ┌─────────────────┐
                          │   Login Form    │
                          │  or API POST    │
                          └─────────────────┘
                                   │
                                   ▼
                       ┌──────────────────────┐
                       │  AuthenticationManager│
                       │  (Spring Security)    │
                       └──────────────────────┘
                                   │
                       ┌───────────┴───────────┐
                       │                       │
                       ▼                       ▼
             ┌────────────────────┐   ┌────────────────────┐
             │   Web Session      │   │   JWT Token        │
             │   (Thymeleaf)      │   │   (REST API)       │
             └────────────────────┘   └────────────────────┘
```

Authorization Matrix Implementation

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain apiSecurityChain(HttpSecurity http) {
        return http
            .securityMatcher("/api/**")
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/**").permitAll()
                .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtFilter, UsernamePasswordAuthFilter.class)
            .build();
    }
}
```

Password Security

- **Hashing:** BCrypt with cost factor 12 (~250ms hash time)
- **Requirements:** 8+ chars, upper, lower, digit, special
- **Storage:** Only hashed passwords stored; never log passwords

JWT Security

- **Algorithm:** HS256 (HMAC with SHA-256)
- **Secret:** 256-bit key from environment variable
- **Claims:** userId, email, role, issuedAt, expiration
- **Validation:** Signature, expiration, issuer checked on every request

---

Integration Patterns

OCR Service Integration

Interface Contract:

```
┌──────────────────────────────────────────────────────────┐
│                    OCR Service Contract                  │
├──────────────────────────────────────────────────────────┤
│ Endpoint: POST /ocr/appraisal                            │
│ Input:    multipart/form-data with 'file' field (PDF)    │
│ Output:   application/json                               │
│ Timeout:  30 seconds                                     │
├──────────────────────────────────────────────────────────┤
│ Success Response:                                        │
│ {                                                        │
│   "success": true,                                       │
│   "confidenceScore": 0.85,                               │
│   "extractedFields": { ... },                            │
│   "checkboxes": { ... }                                  │
│ }                                                        │
├──────────────────────────────────────────────────────────┤
│ Error Response:                                          │
│ {                                                        │
│   "success": false,                                      │
│   "error": "ERROR_CODE",                                 │
│   "message": "Human readable description"                │
│ }                                                        │
└──────────────────────────────────────────────────────────┘
```

Client Implementation:

```java
@Service
public class OcrClient {
    private final WebClient webClient;

    public OcrResult processAppraisal(Path pdfPath) {
        return webClient.post()
            .uri("/ocr/appraisal")
            .contentType(MULTIPART_FORM_DATA)
            .body(fromResource(new FileSystemResource(pdfPath)))
            .retrieve()
            .onStatus(HttpStatus::isError, this::handleError)
            .bodyToMono(OcrResponse.class)
            .timeout(Duration.ofSeconds(30))
            .map(this::toOcrResult)
            .block();
    }
}
```

Scalability Considerations

Phase 1 Constraints (Acceptable for MVP)

Component	Current Limit	Impact
File Storage	Local disk	Single server only
Sessions	In-memory	Lost on restart
OCR Processing	Synchronous	Blocks upload request
Database	Single instance	No read replicas

Phase 2+ Migrations

Component	Current	Future	Migration Path
Files	Local	S3/GCS	Add StorageAdapter interface
Sessions	Memory	Redis	Configure Spring Session
OCR	Sync	Async	Add message queue (RabbitMQ/SQS)
Database	Single	Replicas	Use read-replica for list queries

Horizontal Scaling Blueprint

```
                    ┌──────────────┐
                    │ Load Balancer │
                    └──────┬───────┘
                           │
          ┌────────────────┼────────────────┐
          │                │                │
   ┌──────▼──────┐   ┌──────▼──────┐   ┌──────▼──────┐
   │  Backend    │   │  Backend    │   │  Backend    │
   │  Instance   │   │  Instance   │   │  Instance   │
   └──────┬──────┘   └──────┬──────┘   └──────┬──────┘
          │                │                │
          └────────────────┼────────────────┘
                           │
          ┌────────────────┼────────────────┐
          │                │                │
   ┌──────▼──────┐   ┌──────▼──────┐   ┌──────▼──────┐
   │   Redis     │   │  Postgres   │   │     S3      │
   │  (Session)  │   │ (Primary)   │   │  (Files)    │
   └─────────────┘   └─────────────┘   └─────────────┘
```

Technology Rationale Recap

Choice	Why This	Why Not Alternatives
Spring Boot 3	Mature, well-documented, team expertise	Quarkus/Micronaut: learning curve
Thymeleaf	No separate frontend build, fast iteration	React: takes longer to set up
PostgreSQL	JSONB for OCR results, robust, free	MySQL: weaker JSON support
Python + FastAPI	Best OCR libs, fast to write	Node: Tesseract binding issues
JWT	Stateless API auth, standard	OAuth2: overkill for internal app
Local file storage	Simple for Phase 1	S3: adds complexity early

---

For implementation details, see IMPLEMENTATION_PLAN.md (./IMPLEMENTATION_PLAN.md)

API Reference

Complete REST API documentation for the Ardur Appraisal Management System.

Base URL: `http://localhost:8080/api/v1`

Content-Type: `application/json` (unless otherwise noted)

Authentication: Bearer token in `Authorization` header

Table of Contents

1. Authentication
2. Appraisals

3. Admin
4. Error Responses

Authentication

Login

Authenticate with email and password to receive JWT tokens.

```
POST /auth/login
```

Request Body:

```
{
  "email": "underwriter@example.com",
  "password": "SecurePass123!"
}
```

Success Response (200):

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "user": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "underwriter@example.com",
    "firstName": "Jane",
    "lastName": "Smith",
    "role": "UNDERWRITER"
  }
}
```

Error Response (401):

```
{
  "error": "INVALID_CREDENTIALS",
  "message": "Email or password is incorrect",
  "timestamp": "2025-12-10T10:30:00Z"
}
```

Current User

Get information about the authenticated user.

```
GET /auth/me
Authorization: Bearer <accessToken>
```

Success Response (200):

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "email": "underwriter@example.com",
  "firstName": "Jane",
  "lastName": "Smith",
  "role": "UNDERWRITER",
  "isActive": true,
  "createdAt": "2025-11-15T10:00:00Z"
}
```

Refresh Token

Get a new access token using a refresh token.

```
POST /auth/refresh
```

Request Body:

```
{
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Success Response (200):

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "tokenType": "Bearer",
  "expiresIn": 900
}
```

Appraisals

Upload Appraisal

Upload a new appraisal PDF document for processing.

```
POST /appraisals/upload
Authorization: Bearer <accessToken>
Content-Type: multipart/form-data
```

Form Fields:

Field	Type	Required	Description
file	File	Yes	PDF file, max 50MB
loanNumber	String	No	Loan reference number

cURL Example:

```
curl -X POST http://localhost:8080/api/v1/appraisals/upload \
  -H "Authorization: Bearer eyJhbGci..." \
  -F "file=@/path/to/appraisal.pdf" \
  -F "loanNumber=LN-2025-12345"
```

Success Response (201):

```
{
  "id": "660e8400-e29b-41d4-a716-446655440001",
  "status": "PENDING",
  "originalFilename": "AppraisalReport_1234.pdf",
  "loanNumber": "LN-2025-12345",
  "assignedTo": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "underwriter@example.com"
  },
  "ocrStatus": "PROCESSING",
  "createdAt": "2025-12-10T14:35:00Z",
  "message": "Appraisal uploaded successfully. OCR processing initiated."
}
```

Error Response (400):

```
{
  "error": "INVALID_FILE_TYPE",
  "message": "Only PDF files are accepted. Received: image/jpeg",
  "timestamp": "2025-12-10T14:35:00Z"
}
```

List Appraisals

Get a paginated list of appraisals with optional filtering.

```
GET /appraisals
Authorization: Bearer <accessToken>
```

Query Parameters:

Parameter	Type	Default	Description
status	String	(all)	Filter: PENDING, APPROVED, NEEDS_REVISION
page	Integer	0	Page number (zero-indexed)
size	Integer	20	Items per page (max 100)
sortBy	String	createdAt	Sort field: createdAt, borrowerName, appraisedValue
sortDir	String	desc	Sort direction: asc, desc

Example Request:

```
GET /appraisals?status=PENDING&page=0&size=10&sortBy=createdAt&sortDir=desc
```

Success Response (200):

```
{
  "content": [
    {
      "id": "660e8400-e29b-41d4-a716-446655440001",
      "borrowerName": "John Doe",
      "propertyAddress": "123 Main Street, Anytown, NY 12345",
      "appraisedValue": 450000.00,
      "status": "PENDING",
      "formType": "1004",
      "loanNumber": "LN-2025-12345",
      "qcIssueCount": 2,
      "qcHighSeverityCount": 0,
      "assignedTo": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "email": "underwriter@example.com",
        "firstName": "Jane"
      },
      "createdAt": "2025-12-10T14:35:00Z",
      "updatedAt": "2025-12-10T14:36:00Z"
    }
  ],
  "page": 0,
  "size": 10,
  "totalElements": 45,
  "totalPages": 5,
  "first": true,
  "last": false
}
```

Get Appraisal Details

Get complete information for a specific appraisal, including OCR results and QC issues.

```
GET /appraisals/{id}
Authorization: Bearer <accessToken>
```

Path Parameters:

Parameter	Type	Description
id	UUID	Appraisal ID

Success Response (200):

```
{
  "id": "660e8400-e29b-41d4-a716-446655440001",
  "originalFilename": "AppraisalReport_1234.pdf",
  "fileUrl": "/api/v1/appraisals/660e8400-e29b-41d4-a716-446655440001/file",
  "borrowerName": "John Doe",
  "coBorrowerName": null,
  "propertyAddress": "123 Main Street, Anytown, NY 12345",
  "city": "Anytown",
  "state": "NY",
  "zipCode": "12345",
  "appraisedValue": 450000.00,
  "salePrice": 440000.00,
  "loanNumber": "LN-2025-12345",
  "formType": "1004",
  "status": "PENDING",
  "assignedTo": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "underwriter@example.com",
    "firstName": "Jane",
    "lastName": "Smith"
  },
  "ocrResult": {
    "id": "770e8400-e29b-41d4-a716-446655440000",
    "processingStatus": "COMPLETED",
    "confidenceScore": 0.87,
    "extractedFields": {
      "borrowerName": "John Doe",
      "propertyAddress": "123 Main Street, Anytown, NY 12345",
      "appraisedValue": 450000,
      "effectiveDate": "2025-12-01",
      "lenderName": "ABC Mortgage Corp",
      "appraiserName": "Licensed Appraiser Inc."
    },
    "checkboxes": {
      "isInFloodZone": false,
      "isForSale": false,
      "hasPoolOrSpa": true,
      "isCondoOrPUD": false
    },
    "warnings": [
      "Low confidence on 'appraiserLicenseNumber' field (0.65)"
    ],
    "createdAt": "2025-12-10T14:36:00Z"
  },
  "qcIssues": [
    {
      "id": "880e8400-e29b-41d4-a716-446655440001",
      "ruleCode": "RULE-003",
      "severity": "MEDIUM",
      "fieldName": "propertyAddress",
      "message": "Property address does not include ZIP+4 code",
      "isResolved": false,
      "createdAt": "2025-12-10T14:36:00Z"
    },
    {
      "id": "880e8400-e29b-41d4-a716-446655440002",
      "ruleCode": "RULE-005",
      "severity": "LOW",
      "fieldName": null,
      "message": "OCR confidence below 90% threshold - recommend manual verification",
      "isResolved": false,
      "createdAt": "2025-12-10T14:36:00Z"
    }
  ],
  "reviewHistory": [
```

```
    {
      "id": "990e8400-e29b-41d4-a716-446655440001",
      "user": {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "email": "underwriter@example.com",
        "firstName": "Jane"
      },
      "oldStatus": null,
      "newStatus": "PENDING",
      "comment": "Initial upload",
      "createdAt": "2025-12-10T14:35:00Z"
    }
  ],
  "createdAt": "2025-12-10T14:35:00Z",
  "updatedAt": "2025-12-10T14:36:00Z"
}
```

Error Response (404):

```
{
  "error": "NOT_FOUND",
  "message": "Appraisal not found with id: 660e8400-e29b-41d4-a716-446655440001",
  "timestamp": "2025-12-10T14:40:00Z"
}
```

Download Appraisal PDF

Download the original PDF file.

```
GET /appraisals/{id}/file
Authorization: Bearer <accessToken>
```

Response: Binary PDF file

Headers:

```
Content-Type: application/pdf
Content-Disposition: attachment; filename="AppraisalReport_1234.pdf"
```

Change Appraisal Status

Update the status of an appraisal with a required comment.

```
POST /appraisals/{id}/status
Authorization: Bearer <accessToken>
```

Request Body:

```
{
  "status": "APPROVED",
  "comment": "All fields verified against loan documentation. Value is consistent with comparable sales in the area. QC issues revie
}
```

Valid Status Transitions:

From	To	Allowed
PENDING	APPROVED	✓
PENDING	NEEDS_REVISION	✓
APPROVED	PENDING	✗
APPROVED	NEEDS_REVISION	✗
NEEDS_REVISION	PENDING	✗
NEEDS_REVISION	APPROVED	✗

Success Response (200):

```
{
  "id": "660e8400-e29b-41d4-a716-446655440001",
  "status": "APPROVED",
  "previousStatus": "PENDING",
  "updatedAt": "2025-12-10T15:00:00Z",
  "reviewAction": {
    "id": "990e8400-e29b-41d4-a716-446655440002",
    "oldStatus": "PENDING",
    "newStatus": "APPROVED",
    "comment": "All fields verified against loan documentation...",
    "user": {
      "email": "underwriter@example.com",
      "firstName": "Jane"
    },
    "createdAt": "2025-12-10T15:00:00Z"
  }
}
```

Error Response (400 - Invalid Transition):

```
{
  "error": "INVALID_STATUS_TRANSITION",
  "message": "Cannot transition from APPROVED to PENDING. Approved appraisals are final.",
  "timestamp": "2025-12-10T15:00:00Z"
}
```

Error Response (400 - Missing Comment):

```
{
  "error": "VALIDATION_ERROR",
  "message": "Comment is required when changing status",
  "field": "comment",
  "timestamp": "2025-12-10T15:00:00Z"
}
```

Retry OCR Processing

Manually trigger OCR processing for an appraisal that failed or was not processed.

```
POST /appraisals/{id}/process
Authorization: Bearer <accessToken>
```

Success Response (200):

```
{
  "id": "660e8400-e29b-41d4-a716-446655440001",
  "ocrStatus": "COMPLETED",
  "message": "OCR processing completed successfully",
  "extractedFields": {
    "borrowerName": "John Doe",
    "propertyAddress": "123 Main Street, Anytown, NY 12345",
    "appraisedValue": 450000
  }
}
```

Error Response (503 - OCR Service Unavailable):

```
{
  "error": "OCR_SERVICE_UNAVAILABLE",
  "message": "OCR service is currently unavailable. Please try again later.",
  "timestamp": "2025-12-10T15:00:00Z"
}
```

Admin

> ***Note:*** *All admin endpoints require* `ROLE_ADMIN`*. Underwriters will receive 403 Forbidden.*

List Users

Get all users in the system.

```
GET /admin/users
Authorization: Bearer <accessToken>
```

Success Response (200):

```
{
  "content": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "email": "admin@example.com",
      "firstName": "Admin",
      "lastName": "User",
      "role": "ADMIN",
      "isActive": true,
      "appraisalCount": 0,
      "createdAt": "2025-11-01T10:00:00Z",
      "lastLoginAt": "2025-12-10T09:00:00Z"
    },
    {
      "id": "550e8400-e29b-41d4-a716-446655440010",
      "email": "underwriter@example.com",
      "firstName": "Jane",
      "lastName": "Smith",
      "role": "UNDERWRITER",
      "isActive": true,
      "appraisalCount": 23,
      "createdAt": "2025-11-15T10:00:00Z",
      "lastLoginAt": "2025-12-10T08:30:00Z"
    }
  ],
  "totalElements": 2
}
```

Create User

Create a new user account.

```
POST /admin/users
Authorization: Bearer <accessToken>
```

Request Body:

```
{
  "email": "new.underwriter@example.com",
  "firstName": "New",
  "lastName": "User",
  "role": "UNDERWRITER",
  "temporaryPassword": "TempPass123!"
}
```

Success Response (201):

```
{
  "id": "550e8400-e29b-41d4-a716-446655440020",
  "email": "new.underwriter@example.com",
  "firstName": "New",
  "lastName": "User",
  "role": "UNDERWRITER",
  "isActive": true,
  "createdAt": "2025-12-10T16:00:00Z",
  "message": "User created successfully. User must change password on first login."
}
```

Error Response (400 - Duplicate Email):

```
{
  "error": "DUPLICATE_EMAIL",
  "message": "User with email 'new.underwriter@example.com' already exists",
  "timestamp": "2025-12-10T16:00:00Z"
}
```

Update User Role

Change a user's role.

```
PATCH /admin/users/{id}/role
Authorization: Bearer <accessToken>
```

Request Body:

```
{
  "role": "ADMIN"
}
```

Success Response (200):

```
{
  "id": "550e8400-e29b-41d4-a716-446655440010",
  "email": "underwriter@example.com",
  "role": "ADMIN",
  "previousRole": "UNDERWRITER",
  "updatedAt": "2025-12-10T16:30:00Z"
}
```

Deactivate User

Soft-delete a user (preserves audit trail, prevents login).

```
DELETE /admin/users/{id}
Authorization: Bearer <accessToken>
```

Success Response (200):

```
{
  "id": "550e8400-e29b-41d4-a716-446655440010",
  "email": "underwriter@example.com",
  "isActive": false,
  "message": "User deactivated successfully"
}
```

Error Response (400 - Self-Delete):

```
{
  "error": "CANNOT_DEACTIVATE_SELF",
  "message": "You cannot deactivate your own account",
  "timestamp": "2025-12-10T16:30:00Z"
}
```

Error Responses

All error responses follow this structure:

```
{
  "error": "ERROR_CODE",
  "message": "Human-readable description",
  "field": "fieldName",
  "timestamp": "2025-12-10T10:30:00Z"
}
```

HTTP Status Codes

Code	Meaning	When Used
200	OK	Successful GET, PUT, PATCH, DELETE
201	Created	Successful POST (resource created)
400	Bad Request	Validation errors, invalid input
401	Unauthorized	Missing or invalid authentication
403	Forbidden	Authenticated but not authorized
404	Not Found	Resource doesn't exist
413	Payload Too Large	File exceeds 50MB limit
415	Unsupported Media Type	Wrong file type
500	Internal Server Error	Unexpected server error
503	Service Unavailable	OCR service down

Common Error Codes

Code	Description
INVALID_CREDENTIALS	Wrong email or password
TOKEN_EXPIRED	JWT access token has expired
TOKEN_INVALID	JWT is malformed or tampered
VALIDATION_ERROR	Request body failed validation
NOT_FOUND	Requested resource doesn't exist
ACCESS_DENIED	User lacks required permission
INVALID_FILE_TYPE	Uploaded file is not a PDF
FILE_TOO_LARGE	File exceeds size limit
INVALID_STATUS_TRANSITION	Status change not allowed
OCR_SERVICE_UNAVAILABLE	Python OCR service is down
DUPLICATE_EMAIL	Email already registered

Rate Limiting

Phase 1 does not implement rate limiting. Future phases may add:

- 100 requests per minute per user
- 10 file uploads per minute per user
- 429 Too Many Requests response when exceeded

For architecture details, see ARCHITECTURE.md (./ARCHITECTURE.md)

For implementation timeline, see IMPLEMENTATION_PLAN.md (./IMPLEMENTATION_PLAN.md)